

Ocelo

Manual técnico

Instalação, publicação no GitHub Pages, uso de cada parte da interface, arquitetura do código, motor de quadros byte a byte, formato de dados dos protocolos, testes, segurança, personalização e solução de problemas.

Versão 1.0.1
Novembro de 2026

Copyright (©) 2026 Erik Pimenta

Sumário

01 **Apresentação** **4**

1.1 O que é o Ocelo 4
1.2 Para quem é 4
1.3 Tecnologias 5
1.4 Licença de uso 5
1.5 Estrutura de pastas 5

02 **Instalação** **6**

2.1 Requisitos 6
2.2 Rodando na sua máquina 6
2.3 Gerando a versão de produção 6
2.4 Publicando no GitHub Pages 6
2.5 Atualizando uma instalação 7

03 **Usando a interface** **8**

3.1 Visão geral da tela 8
3.2 Lista de protocolos e abas 8
3.3 Camadas OSI 8
3.4 Vista de rede 8
3.5 Vista de sequência 9
3.6 Reprodução 10
3.7 Detalhes do passo 10
3.8 Fluxo de pacotes 11
3.9 Inspetor byte a byte 11
3.10 Busca e atalhos 12
3.11 Idiomas e temas 13
3.12 Ritmo das animações 13
3.13 Modo foco e painéis 14
3.14 Endereços por protocolo 14

04 **Arquitetura** **15**

4.1 Visão geral 15
4.2 Estado e reprodutor 15
4.3 Linha do tempo 15
4.4 Motor de quadros 15
4.5 Rotas e topologia 16
4.6 Interface e animações 16

Copyright (©) 2026 Erik Pimenta

4.7 Idiomas 16

05 **Dados de um protocolo** **17**

5.1 O contrato 17

5.2 Pacotes e quadros 17

5.3 Adicionando um protocolo (mantenedor) 18

06 **Os 33 protocolos** **19**

6.1 Catálogo 19

6.2 Ficha de cada protocolo 21

07 **Qualidade e segurança** **34**

7.1 Testes 34

7.2 Integração contínua 34

7.3 Segurança 34

7.4 Acessibilidade 34

08 **Personalização** **36**

8.1 Cores e temas 36

8.2 Tipografia 36

8.3 Marca 36

09 **Solução de problemas** **37**

9.1 Problemas comuns 37

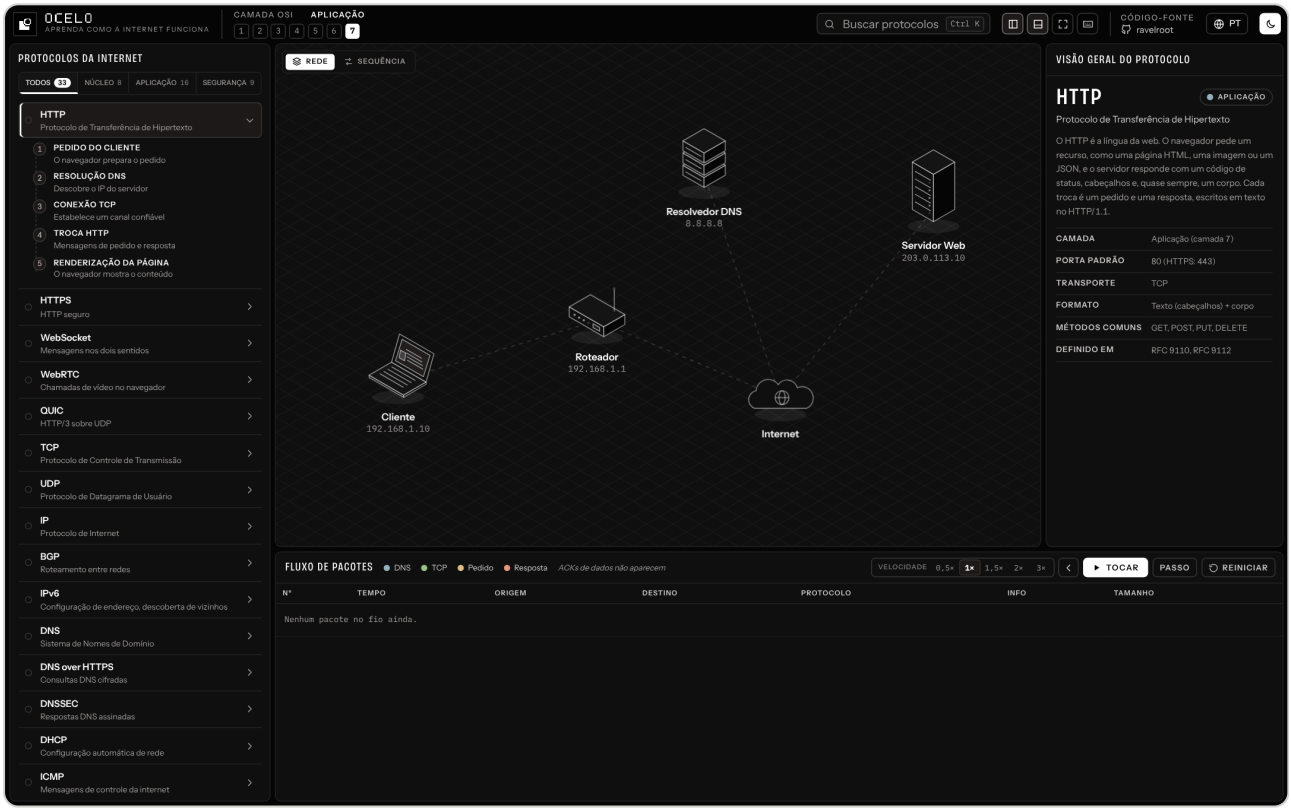
Copyright (©) 2026 Erik Pimenta

01 Apresentação

1.1 O que é o Ocelo

O Ocelo é um explorador visual de protocolos da internet. Ele mostra, passo a passo e pacote a pacote, o que acontece de fato na rede quando alguém abre um site, resolve um nome, entra numa rede Wi-Fi, manda um e-mail ou liga uma VPN. São 33 protocolos, 177 passos e 432 pacotes, cada um montado byte a byte.

A aplicação roda inteira no navegador, sem servidor, e pode ser publicada no GitHub Pages ou em qualquer hospedagem de arquivos estáticos. Todo o conteúdo existe em português e inglês.



Visão geral do HTTP: lista de protocolos, topologia, visão geral do protocolo e fluxo de pacotes.

1.2 Para quem é

- **Estudantes e professores** de redes, que precisam ver o aperto de mão do TCP, a cadeia do DNS ou o 4-way handshake do Wi-Fi acontecendo, e não só ler sobre eles.
- **Pessoas de desenvolvimento e operação** que querem entender o que está por trás de HTTPS, proxies, WebSocket, QUIC ou WireGuard antes de depurar um problema.
- **Quem se prepara para entrevistas e certificações**, com cada passo explicado e os números que importam (portas, flags, tamanhos e RFCs).

Copyright (©) 2026 Erik Pimenta

1.3 Tecnologias

Tecnologia	Papel
TypeScript 7 (modo estrito)	Toda a aplicação e os dados dos protocolos
Vite 8	Servidor de desenvolvimento e build estático
SVG e CSS	Topologia isométrica, sequência, animações e temas claro e escuro
Vitest 5	Testes unitários do catálogo, dos quadros e dos checksums
Playwright	Testes de ponta a ponta da interface
GitHub Actions	CI, auditoria de dependências e publicação no GitHub Pages
Fontsource	Bricolage Grotesque, Instrument Sans e IBM Plex Mono servidas pelo próprio site

1.4 Licença de uso

Copyright (©) 2026 Erik Pimenta. Todos os direitos reservados. A partir da versão 1.0.1, qualquer pessoa pode usar a ferramenta publicada, gratuitamente. Copiar, modificar, redistribuir, hospedar em outro endereço ou reutilizar o código, os textos, os dados dos protocolos, os desenhos, o nome ou a logo depende de autorização por escrito do autor. O texto completo está no arquivo `LICENSE`; os componentes de terceiros seguem as próprias licenças.

1.5 Estrutura de pastas

```
src/
  app/          estado, reprodutor e linha do tempo
  css/          estilo.css: tokens, layout, componentes e animações
  dados/
    protocolos/ os 33 protocolos e o catálogo (index.ts)
    tipos.ts    contrato de dados de um protocolo
  i18n/         textos da interface em português e inglês
  rede/         motor de quadros (quadros.ts) e rotas (rotas.ts)
  ui/           componentes da interface e desenhos isométricos
  main.ts       ponto de entrada e atalhos de teclado
public/         ícone do site
tests/unit      Vitest
tests/e2e       Playwright
docs/           manuais em PDF e imagens do README
.github/        CI, GitHub Pages e Dependabot
```

02 Instalação

2.1 Requisitos

- Node.js 22 ou mais novo (o arquivo `.nvmrc` indica a versão para o nvm e o fnm).
- npm 10 ou mais novo, que já vem com o Node.js.
- Um navegador atual (Chrome, Edge, Firefox ou Safari).

2.2 Rodando na sua máquina

1. Clone o repositório: `git clone https://github.com/ravelroot/Network-Protocol-Packet-Explorer.git`.
2. Entre na pasta: `cd Network-Protocol-Packet-Explorer`.
3. Instale as dependências exatamente como estão no lockfile: `npm ci`.
4. Suba o servidor de desenvolvimento: `npm run dev`.
5. Abra `http://localhost:5173`. Cada alteração no código recarrega a página sozinha.

2.3 Gerando a versão de produção

1. Rode `npm run build`. O TypeScript é conferido antes e o resultado vai para a pasta `dist/`.
2. Confira o resultado com `npm run preview` e abra `http://localhost:4173`.
3. Copie a pasta `dist/` para qualquer servidor de arquivos estáticos. Os caminhos são relativos, então ela funciona também dentro de uma subpasta.

> **POLÍTICA DE SEGURANÇA**
> O build injeta no `index.html` uma política de segurança de conteúdo que só permite scripts, estilos e fontes do próprio site. Se você servir a pasta por um proxy que adicione scripts, ajuste a política em `vite.config.ts`.

2.4 Publicando no GitHub Pages

1. Envie o projeto para a branch `main` do repositório `ravelroot/Network-Protocol-Packet-Explorer` no GitHub.
2. No repositório, abra **Settings** e depois **Pages**.

Copyright (©) 2026 Erik Pimenta

3. Em **Build and deployment**, escolha **GitHub Actions** como fonte.
4. Faça qualquer push na branch `main`. O workflow **GitHub Pages** instala as dependências, roda os testes, gera o build e publica.
5. Abra o endereço mostrado em **Pages**: `https://ravelroot.github.io/Network-Protocol-Packet-Explorer/`.

2.5 Atualizando uma instalação

1. Baixe as mudanças: `git pull`.
2. Reinstale as dependências: `npm ci`.
3. Confira tudo: `npm run typecheck` e `npm test`.
4. Gere de novo: `npm run build`. No GitHub Pages, o push na `main` já faz esse passo.

Copyright (©) 2026 Erik Pimenta

03 Usando a interface

3.1 Visão geral da tela

A tela tem cinco áreas: o cabeçalho no topo, a lista de protocolos à esquerda, o palco no centro (rede, sequência ou inspetor), o painel de detalhes à direita e o fluxo de pacotes embaixo. Em telas menores as áreas se empilham na mesma ordem.

Área	O que mostra
Cabeçalho	Marca, camada OSI do protocolo aberto, busca, botões de painéis, código-fonte, idioma e tema
Lista	Abas por categoria, protocolos e os passos do protocolo aberto
Palco	Vista de rede, vista de sequência ou o inspetor de um pacote
Painel	Visão geral do protocolo ou detalhes do passo atual
Fluxo de pacotes	Legenda, controles de reprodução e a tabela de pacotes

3.2 Lista de protocolos e abas

As abas **Todos**, **Núcleo**, **Aplicação** e **Segurança** filtram a lista e mostram quantos protocolos há em cada uma (33, 8, 16 e 9). Um clique num protocolo abre os passos dele logo abaixo. Cada passo mostra o número, o título e um resumo; os passos já feitos ganham um sinal de confirmado e o atual fica destacado.

1. Clique em um protocolo para abri-lo. Clicar de novo no mesmo protocolo recolhe os passos.
2. Clique em um passo para ir direto ao fim dele: todos os pacotes até ali aparecem de uma vez.
3. Use `↑` e `↓` para ir ao protocolo anterior ou seguinte, respeitando o filtro da aba.

3.3 Camadas OSI

No cabeçalho, os botões de 1 a 7 representam as camadas do modelo OSI. O botão da camada do protocolo aberto fica preenchido e o nome dela aparece acima (por exemplo, **Transporte** para o TCP). Um clique em qualquer número filtra a lista para os protocolos daquela camada; o filtro aparece acima da lista e some com o botão de fechar.

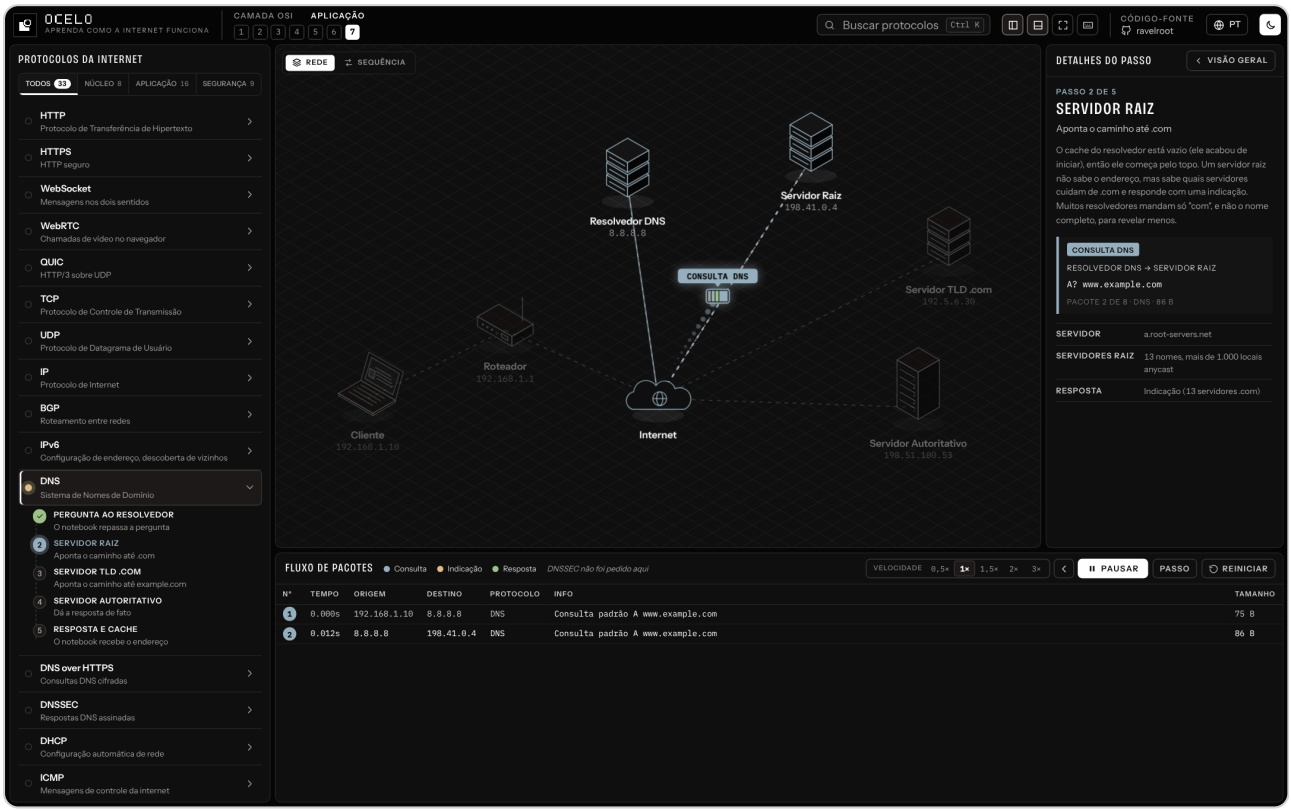
3.4 Vista de rede

A vista de rede desenha os aparelhos em estilo isométrico, com o nome e o endereço de cada um, e as ligações entre eles. Durante a reprodução, cada pacote sai do aparelho de origem com uma onda, viaja pelo caminho real (notebook, roteador, internet, servidor), faz uma pequena parada em cada salto e chega com uma onda

Copyright (©) 2026 Erik Pimenta

mais forte. O pacote mostra o rótulo e as faixas das camadas que carrega, deixa um rastro e acende o trilho na cor do tipo. Quando um pacote se perde, ele para no meio do caminho e cai, com um X.

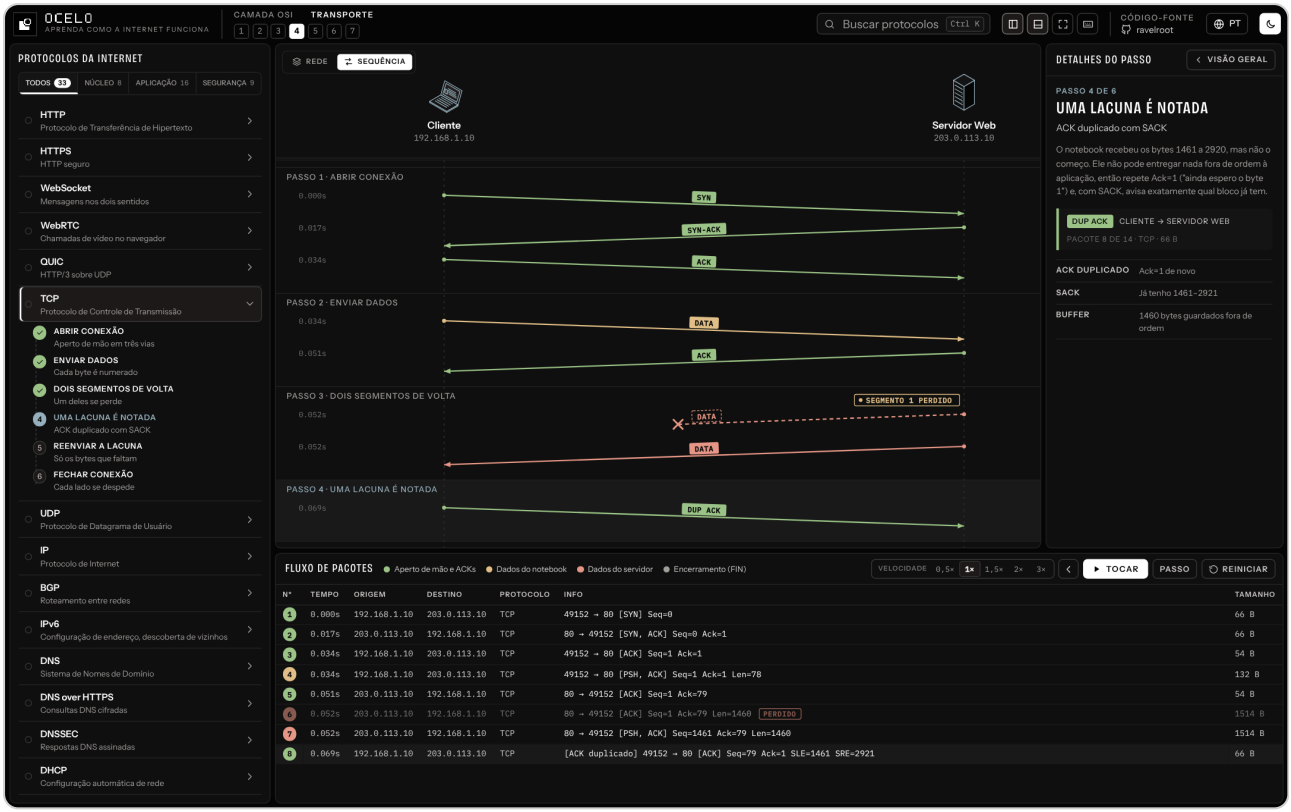
Os aparelhos e ligações que não participam do passo atual ficam esmaecidos, para o olho seguir só o que importa.



DNS em reprodução: a consulta do resolvedor viaja até o servidor raiz.

3.5 Vista de sequência

A vista de sequência coloca as pontas da conversa em faixas verticais, com o ícone, o nome e o endereço fixos no topo. Cada passo é uma faixa horizontal e cada pacote é uma seta inclinada, desenhada no ritmo da reprodução, com o rótulo e o tempo. Notas como "fora do cache" ou "segmento 1 perdido" aparecem junto da seta. Use o botão **Sequência** ou a tecla `V`.



TCP: o segmento perdido, o ACK duplicado com SACK e a retransmissão.

3.6 Reprodução

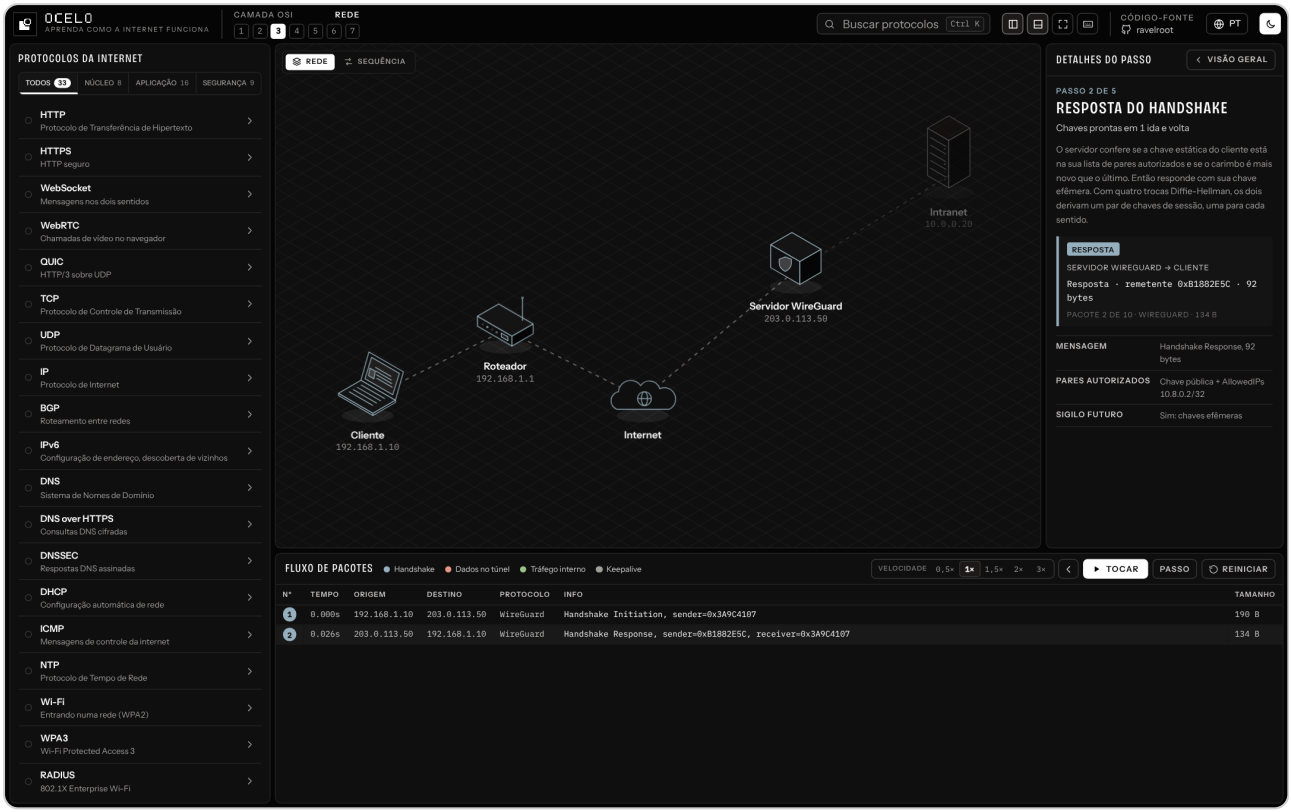
Controle	Função
Tocar / Pausar	Reproduz os passos em sequência, pacote a pacote (tecla Espaço)
Passo	Completa o passo atual ou avança para o próximo (seta para a direita)
Voltar	Volta para o fim do passo anterior (seta para a esquerda)
Reiniciar	Limpa o fio e volta à visão geral (tecla R)
Velocidade	0,5×, 1×, 1,5×, 2× ou 3× (teclas − e +); vale na hora e fica salva

Passos sem pacotes, como a renderização da página no HTTP, acontecem dentro do aparelho: o painel explica o que ocorre e a reprodução segue depois de uma pausa maior.

3.7 Detalhes do passo

O painel da direita começa com a visão geral do protocolo: sigla, nome, categoria, descrição e uma tabela com camada, porta, transporte, formato e RFCs. Quando um passo começa, o painel passa a mostrar **Passo N de M**, o título, o resumo, a explicação, o cartão do último pacote (rótulo, origem e destino, conteúdo e tamanho) e os números daquele passo. O botão **Visão geral** volta para o resumo do protocolo, e um clique no cartão abre o pacote no inspetor.

Copyright (©) 2026 Erik Pimenta



Detalhes de um passo do WireGuard.

3.8 Fluxo de pacotes

A tabela segue o estilo do Wireshark: número, tempo desde o primeiro pacote, origem, destino, protocolo, informação e tamanho do quadro. As cores dos números seguem a legenda do protocolo, que aparece no topo junto com observações (por exemplo, "ACKs de dados não aparecem"). Pacotes perdidos ficam esmaecidos e marcados. Um clique ou `Enter` numa linha abre o pacote no inspetor.

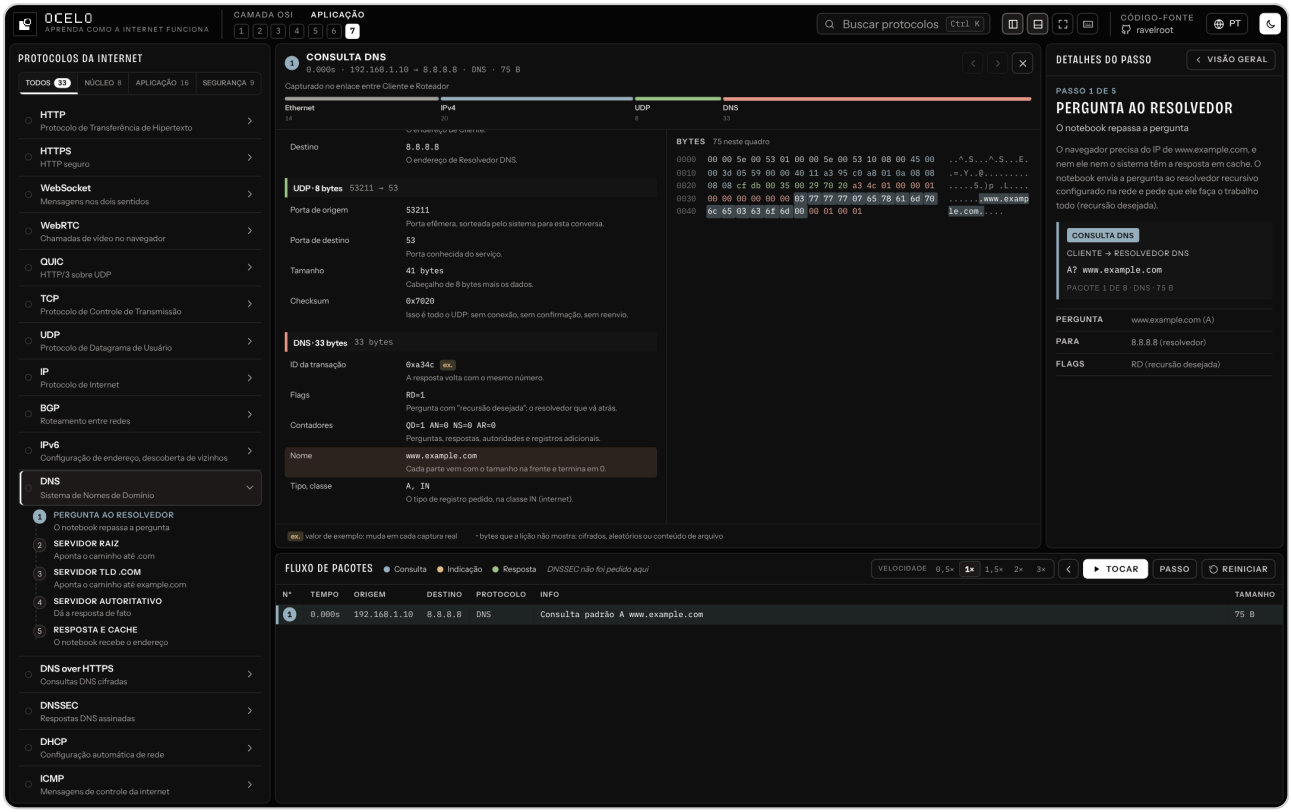
3.9 Inspetor byte a byte

O inspetor mostra o quadro inteiro de um pacote. No topo ficam o número, o rótulo, o tempo, os endereços, o protocolo e o tamanho, além de onde o quadro foi capturado. A barra de camadas mostra o tamanho proporcional de cada camada (por exemplo, Ethernet 14, IPv4 20, UDP 8 e DNS 33 bytes).

À esquerda, cada camada lista os campos com o valor e uma explicação curta. À direita, o dump hexadecimal mostra os bytes e o texto ASCII. Passe o mouse ou o foco do teclado num campo para acender os bytes dele; passe num byte para acender o campo.

Marca	Significado
ex.	Valor de exemplo, que muda em toda captura real (endereços MAC, identificação IP, IDs)
-	Bytes que a lição não mostra: cifrados, aleatórios ou conteúdo de arquivo

Copyright (©) 2026 Erik Pimenta



Inspetor da consulta DNS: 75 bytes, como numa captura real.

3.10 Busca e atalhos

`Ctrl` + `K` ou `/` abre a busca. Ela procura por sigla, nome e resumo nas duas línguas, ignora acentos e aceita várias palavras. As setas escolhem o resultado e `Enter` abre.

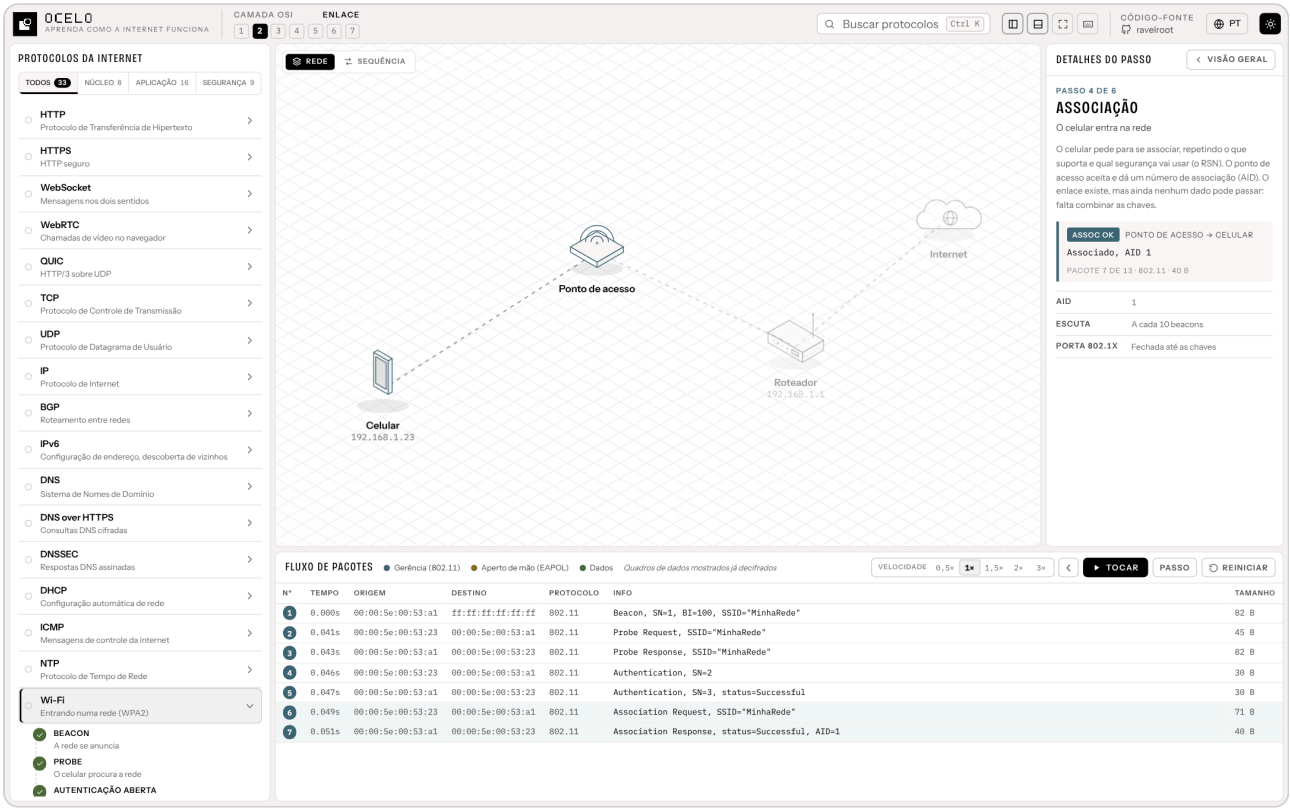
Tecla	Ação
Espaço	Tocar ou pausar
− / +	Mais devagar / mais rápido
→ / ←	Próximo passo / passo anterior
↓ / ↑	Próximo protocolo / anterior
R	Reiniciar
V	Rede ou sequência
Ctrl + K ou /	Buscar
[/]	Lista / fluxo de pacotes
F	Modo foco
T	Tema claro ou escuro

Copyright (©) 2026 Erik Pimenta

Tecla	Ação
L	Português ou inglês
Esc	Fechar inspetor ou janela
?	Folha de atalhos

3.11 Idiomas e temas

O botão com o globo alterna entre português e inglês; na primeira visita o Ocelo segue o idioma do navegador. O botão de lua ou sol alterna entre os temas escuro e claro, os dois na paleta do Bento. Idioma, tema, vista e velocidade ficam salvos no navegador.



Wi-Fi no tema claro.

3.12 Ritmo das animações

O Ocelo é feito para ensinar, então a animação começa devagar: cada pacote leva cerca de três segundos para atravessar a rede, para numa pausa curta em cada roteador e só então segue. Entre um passo e outro há um intervalo para ler o painel da direita.

O controle **Velocidade**, no topo do fluxo de pacotes, oferece 0,5×, 1×, 1,5×, 2× e 3× em um clique, e as teclas `−` e `+` fazem o mesmo. A mudança vale na hora, inclusive no meio de um pacote em voo.

Copyright (©) 2026 Erik Pimenta

3.13 Modo foco e painéis

Os botões do cabeçalho escondem a lista (`[`) e o fluxo de pacotes (`]`). O modo foco (`F`) deixa só o palco na tela, bom para projetar em aula. `Esc` sai do modo foco.

3.14 Endereços por protocolo

Cada protocolo tem o próprio endereço, como `#/dns` ou `#/wireguard`. Abrir esse link leva direto ao protocolo, o que facilita compartilhar uma lição.

Copyright (©) 2026 Erik Pimenta

04 Arquitetura

4.1 Visão geral

O Ocelo é uma aplicação estática em TypeScript, sem framework. Os dados dos protocolos ficam em arquivos TypeScript tipados; a linha do tempo, o reprodutor e o motor de quadros transformam esses dados em estado; e os componentes da interface observam o estado e desenham a tela com o DOM e SVG.

```
dados/protocolos ─┬─> linha do tempo ──> reprodutor ──> loja de estado ──> interface
                  └─> motor de quadros ──────────────────────────────────────> inspetor
reprodutor ── requestAnimationFrame ──> rede e sequência (pacote em voo)
```

4.2 Estado e reprodutor

A `Loja` (`src/app/estado.ts`) guarda o estado da tela: protocolo, filtros, vista, passo atual, quantos pacotes já foram entregues, qual está em voo, velocidade, idioma, tema e painéis. Cada componente assina a loja e redesenha só quando o que ele mostra muda.

O `Reprodutor` (`src/app/reprodutor.ts`) conduz a reprodução: entra em cada passo, anima cada pacote quadro a quadro e avisa os componentes da fração percorrida. O avanço de cada quadro é multiplicado pela velocidade atual, por isso trocar a velocidade muda o ritmo na hora. Cada reprodução tem uma geração; pausar, voltar ou trocar de protocolo invalida a geração anterior, o que cancela as animações pendentes sem condição de corrida.

4.3 Linha do tempo

A linha do tempo (`src/app/linha-do-tempo.ts`) achata os passos numa lista de pacotes com o índice, o passo e o tempo acumulado (o `dt` de cada pacote, em milissegundos). Ela também guarda onde cada passo começa e termina e mantém em cache a análise byte a byte de cada pacote.

4.4 Motor de quadros

O motor (`src/rede/quadros.ts`) monta os bytes de cada pacote a partir da especificação do quadro. Ele acha o caminho pela topologia para saber em qual enlace o quadro foi capturado e quais MACs usar, e então escreve as camadas:

Camada	O que é montado
Enlace	Ethernet II (14 bytes) ou 802.11 (24 bytes) com LLC/SNAP para dados; quadros de gerência Wi-Fi (beacon, probe, autenticação, associação)
Rede	IPv4 com checksum, IPv6, ARP ou EAPOL (802.1X)

Copyright (©) 2026 Erik Pimenta

Camada	O que é montado
Transporte	TCP com opções no SYN (MSS, SACK permitido, escala de janela) e bloco SACK, UDP e ICMP/ICMPv6, todos com checksum e pseudo-cabeçalho
Aplicação	Campos da carga em hex, texto ou bytes opacos, cada um com nome, valor e explicação

Cada campo guarda o intervalo de bytes que ocupa, o que permite acender campo e bytes juntos no inspetor. Bytes opacos usam um gerador determinístico, então o mesmo pacote gera sempre os mesmos bytes.

4.5 Rotas e topologia

As rotas (`src/rede/rotas.ts`) fazem uma busca em largura pelas ligações do protocolo para achar o caminho entre dois nós, com cache por protocolo. A vista de rede usa esse caminho para animar o pacote salto a salto, e o motor usa o primeiro salto para escolher os endereços MAC.

4.6 Interface e animações

Os componentes ficam em `src/ui` e criam cada elemento pelo DOM (`h()` e `s()` em `dom.ts`), sem montar HTML a partir de texto. Os desenhos isométricos são gerados em `dispositivos.ts` a partir de caixas projetadas.

A vista de rede planeja o percurso de cada pacote (`movimento.ts`): o tempo é dividido entre os trechos de acordo com o comprimento, com uma parada curta em cada salto e aceleração suave. Ondas, trilho, rastro e entradas usam animações CSS, e quem prefere menos movimento (`prefers-reduced-motion`) recebe a versão sem animação.

4.7 Idiomas

Os textos da interface ficam em `src/i18n/textos.ts` ; o conteúdo de cada protocolo usa o tipo `Texto` , com `pt` e `en` lado a lado. A função `tr()` escolhe a versão do idioma atual.

Copyright (©) 2026 Erik Pimenta

05 Dados de um protocolo

5.1 O contrato

Todo protocolo é um objeto `Protocolo` (`src/dados/tipos.ts`).

Campo	Conteúdo
id, sigla	Identificador usado no endereço (#/id) e sigla exibida
nome, resumo, descricao	Textos bilíngues da lista e da visão geral
categoria, camada	core, aplicacao ou seguranca; camada OSI de 1 a 7
fatos	Pares rótulo e valor da visão geral
legenda, observacao	Tipos de pacote com nome e cor (anil, oliva, ouro, brasa, neutra) e uma observação
nos, ligacoes, faixas	Aparelhos com tipo, nome, IP e posição; ligações da topologia; ordem das faixas na sequência
passos	Título, resumo, descrição, fatos e pacotes de cada passo

5.2 Pacotes e quadros

Cada pacote tem origem e destino, o protocolo mostrado na tabela, o rótulo da seta, a informação, o tipo da legenda, um resumo opcional, o intervalo `dt` e, se for o caso, `perdido` e uma `nota`. O `quadro` descreve o enlace, a rede, as portas e flags e os campos da carga.

```
{
  de: 'cliente', para: 'resolvedor', proto: 'DNS',
  rotulo: t('CONSULTA DNS', 'DNS QUERY'),
  info: t('Consulta padrão A www.example.com', 'Standard query A www.example.com'),
  tipo: 'dns', dt: 0,
  quadro: {
    udp: { origem: 53211, destino: 53 },
    carga: [hex(t('ID da transação', 'Transaction ID'), 'd8 25', '0xd825'), ...],
  },
}
```

Copyright (©) 2026 Erik Pimenta

5.3 Adicionando um protocolo (mantenedor)

> **LICENÇA**
> Esta seção é para o mantenedor. Copiar ou modificar o código depende de autorização por escrito do autor.

1. Crie `src/dados/protocolos/<id>.ts` exportando um `Protocolo`; use `http.ts` como modelo.
2. Posicione os nós no palco com `x` de 6 a 94 e `y` de 12 a 88, ligue-os e defina as faixas da sequência.
3. Escreva os passos e pacotes nas duas línguas, usando os ajudantes de `base.ts`, `conversa.ts` e `fluxo-tcp.ts`.
4. Inclua o `id` na lista `ORDEM` de `src/dados/protocolos/index.ts`.
5. Rode `npm test`. A validação aponta textos vazios, nós ou faixas inexistentes, tipos sem legenda e quadros que não fecham.

Copyright (©) 2026 Erik Pimenta

06 Os 33 protocolos

6.1 Catálogo

Protocolo	Categoria	Camada	Passos	Pacotes	Definido em
HTTP Protocolo de Transferência de Hipertexto	Aplicação	7 · Aplicação	5	7	RFC 9110, RFC 9112
HTTPS HTTP seguro	Segurança	7 · Aplicação	5	8	RFC 9110, RFC 8446
WebSocket Mensagens nos dois sentidos	Aplicação	7 · Aplicação	6	9	RFC 6455
WebRTC Chamadas de vídeo no navegador	Aplicação	7 · Aplicação	5	14	RFC 8825–8835, W3C WebRTC
QUIC HTTP/3 sobre UDP	Aplicação	4 · Transporte	5	8	RFC 9000, RFC 9001, RFC 9114
TCP Protocolo de Controle de Transmissão	Núcleo	4 · Transporte	6	14	RFC 9293
UDP Protocolo de Datagrama de Usuário	Núcleo	4 · Transporte	4	9	RFC 768
IP Protocolo de Internet	Núcleo	3 · Rede	5	10	RFC 791
BGP Roteamento entre redes	Núcleo	3 · Rede	6	13	RFC 4271
IPv6 Configuração de endereço, descoberta de vizinhos	Núcleo	3 · Rede	6	8	RFC 8200, RFC 4861, RFC 4862
DNS Sistema de Nomes de Domínio	Núcleo	7 · Aplicação	5	8	RFC 1034, RFC 1035
DNS over HTTPS Consultas DNS cifradas	Segurança	7 · Aplicação	6	8	RFC 8484
DNSSEC Respostas DNS assinadas	Segurança	7 · Aplicação	6	10	RFC 4033, 4034, 4035

Copyright (©) 2026 Erik Pimenta

Protocolo	Categoria	Camada	Passos	Pacotes	Definido em
DHCP Configuração automática de rede	Núcleo	7 · Aplicação	5	6	RFC 2131, RFC 2132
ICMP Mensagens de controle da internet	Núcleo	3 · Rede	5	8	RFC 792
NTP Protocolo de Tempo de Rede	Aplicação	7 · Aplicação	5	6	RFC 5905
Wi-Fi Entrando numa rede (WPA2)	Aplicação	2 · Enlace	6	13	IEEE 802.11-2020 (inclui o 802.11i)
WPA3 Wi-Fi Protected Access 3	Segurança	2 · Enlace	6	11	IEEE 802.11-2020, Wi-Fi Alliance (2018)
RADIUS 802.1X Enterprise Wi-Fi	Segurança	7 · Aplicação	5	16	RFC 2865, RFC 3579, IEEE 802.1X
Captive Portal Logins de Wi-Fi de hotel e café	Aplicação	7 · Aplicação	5	17	RFC 8908, RFC 8910, RFC 8952
SMTP Simple Mail Transfer Protocol	Aplicação	7 · Aplicação	5	28	RFC 5321, RFC 6409, RFC 3207
POP3 Post Office Protocol v3	Aplicação	7 · Aplicação	5	20	RFC 1939, RFC 2595
IMAP Internet Message Access	Aplicação	7 · Aplicação	6	19	RFC 9051 (IMAP4rev2), RFC 2177
FTP File Transfer Protocol	Aplicação	7 · Aplicação	6	23	RFC 959, RFC 2428
SMB Compartilhamento de arquivos do Windows	Aplicação	7 · Aplicação	6	17	MS-SMB2 (Microsoft)
SSH Shell seguro	Segurança	7 · Aplicação	6	20	RFC 4251–4254, RFC 8731
TLS/SSL Transport Layer Security	Segurança	6 · Apresentação	6	14	RFC 8446 (TLS 1.3)
HTTP Proxy Proxy de saída e CONNECT	Aplicação	7 · Aplicação	5	18	RFC 9110, RFC 9112
SOCKS5 Proxy Socket Secure	Aplicação	5 · Sessão	5	16	RFC 1928, RFC 1929

Copyright (©) 2026 Erik Pimenta

Protocolo	Categoria	Camada	Passos	Pacotes	Definido em
Reverse Proxy Terminação de TLS	Aplicação	7 · Aplicação	5	20	RFC 9110 (intermediários), RFC 7239
WireGuard Uma VPN moderna	Segurança	3 · Rede	5	10	Whitepaper WireGuard (NDSS 2017); Linux 5.6+
IPsec Túneis VPN entre redes	Segurança	3 · Rede	5	10	RFC 4301, RFC 4303, RFC 7296
Modbus Controle industrial (OT)	Aplicação	7 · Aplicação	5	14	Modbus Application Protocol v1.1b3

6.2 Ficha de cada protocolo

HTTP · Protocolo de Transferência de Hipertexto

Aplicação, camada 7 (Aplicação), 5 passos e 7 pacotes. Passos: Pedido do cliente; Resolução DNS; Conexão TCP; Troca HTTP; Renderização da página.

Fato	Valor
Camada	Aplicação (camada 7)
Porta padrão	80 (HTTPS: 443)
Transporte	TCP
Formato	Texto (cabeçalhos) + corpo
Métodos comuns	GET, POST, PUT, DELETE
Definido em	RFC 9110, RFC 9112

HTTPS · HTTP seguro

Segurança, camada 7 (Aplicação), 5 passos e 8 pacotes. Passos: Conexão TCP; Client Hello; Server Hello; Verificar e concluir; HTTP cifrado.

Fato	Valor
Camada	Aplicação (HTTP sobre TLS)
Porta padrão	443
Transporte	TCP (ou QUIC no HTTP/3)
Segurança	TLS 1.3: AEAD e sigilo futuro
Identidade	Certificado X.509 + SNI

Copyright (©) 2026 Erik Pimenta

Fato	Valor
Definido em	RFC 9110, RFC 8446

WebSocket · WebSocket

Aplicação, camada 7 (Aplicação), 6 passos e 9 pacotes. Passos: Pedido de upgrade; 101 Switching Protocols; Mensagem do navegador; O servidor fala primeiro; Ping e pong; Fechar.

Fato	Valor
Camada	Aplicação (camada 7)
Porta	80 (ws://) ou 443 (wss://)
Transporte	TCP
Abertura	Upgrade do HTTP/1.1 (status 101)
Quadro	2–14 bytes de cabeçalho
Definido em	RFC 6455

WebRTC · Comunicação Web em Tempo Real

Aplicação, camada 7 (Aplicação), 5 passos e 14 pacotes. Passos: Oferta e resposta; Descobrir o IP público; Testes de conectividade; Aperto de mão DTLS; Mídia SRTP.

Fato	Valor
Camada	Aplicação (camada 7)
Transporte	UDP (SRTP; dados via SCTP sobre DTLS)
Travessia de NAT	ICE com STUN e TURN
Segurança	DTLS-SRTP, sempre cifrado
Sinalização	Livre: WebSocket, HTTP…
Definido em	RFC 8825–8835, W3C WebRTC

QUIC · QUIC

Aplicação, camada 4 (Transporte), 5 passos e 8 pacotes. Passos: Initial do cliente; Resposta do servidor; Pedido HTTP/3; Resposta e ACK; Trocar de rede.

Fato	Valor
Camada	Transporte (camada 4), sobre UDP
Porta	UDP 443

Copyright (©) 2026 Erik Pimenta

Fato	Valor
Aperto de mão	TLS 1.3 embutido: 1 RTT, 0-RTT ao retomar
Fluxos	Independentes, sem bloqueio de cabeça de fila
Identidade	IDs de conexão (sobrevivem à troca de rede)
Definido em	RFC 9000, RFC 9001, RFC 9114

TCP · Protocolo de Controle de Transmissão

Núcleo, camada 4 (Transporte), 6 passos e 14 pacotes. Passos: Abrir conexão; Enviar dados; Dois segmentos de volta; Uma lacuna é notada; Reenviar a lacuna; Fechar conexão.

Fato	Valor
Camada	Transporte (camada 4)
Conexão	Aperto de mão em três vias
Confiabilidade	Números de sequência + ACKs
Recuperação	Retransmissão
Cabeçalho	20–60 bytes
Definido em	RFC 9293

UDP · Protocolo de Datagrama de Usuário

Núcleo, camada 4 (Transporte), 4 passos e 9 pacotes. Passos: Só enviar; Atualizações em fluxo; Um datagrama se perde; Porta fechada.

Fato	Valor
Camada	Transporte (camada 4)
Cabeçalho	8 bytes fixos
Conexão	Nenhuma
Garantias	Nenhuma: pode perder, duplicar ou inverter
Usado por	DNS, QUIC, VoIP, jogos, streaming
Definido em	RFC 768

IP · Protocolo de Internet

Núcleo, camada 3 (Rede), 5 passos e 10 pacotes. Passos: Achar o gateway; Montar o pacote; Salto a salto; A resposta volta; Grande demais.

Copyright (©) 2026 Erik Pimenta

Fato	Valor
Camada	Rede (camada 3)
Endereço	IPv4: 32 bits (192.168.1.10)
Cabeçalho	20–60 bytes
Entrega	Melhor esforço, sem garantia
Encaminhamento	Salto a salto, pelo prefixo mais longo
Definido em	RFC 791

BGP · Protocolo de Roteamento de Borda

Núcleo, camada 3 (Rede), 6 passos e 13 pacotes. Passos: Sessão TCP; Apresentação; Anunciar o prefixo; Passar adiante; Manter viva; Retirar a rota.

Fato	Valor
Camada	Roteamento (camada 3), sobre TCP
Porta	179 (TCP)
Tipo	Vetor de caminhos entre ASes
Mensagens	OPEN, UPDATE, KEEPALIVE, NOTIFICATION
Tabela global	Cerca de 1 milhão de prefixos IPv4
Definido em	RFC 4271

IPv6 · Protocolo de Internet, versão 6

Núcleo, camada 3 (Rede), 6 passos e 8 pacotes. Passos: Endereço local; Procura um roteador; Anúncio do roteador; Endereço global; Acha o gateway; Primeiro pacote.

Fato	Valor
Camada	Rede (camada 3)
Endereço	128 bits (2001:db8::/32 é para documentação)
Cabeçalho	40 bytes fixos, sem checksum
Configuração	SLAAC ou DHCPv6
Vizinhos	NDP sobre ICMPv6 (substitui o ARP)
Definido em	RFC 8200, RFC 4861, RFC 4862

Copyright (©) 2026 Erik Pimenta

DNS · Sistema de Nomes de Domínio

Núcleo, camada 7 (Aplicação), 5 passos e 8 pacotes. Passos: Pergunta ao resolvedor; Servidor raiz; Servidor TLD .com; Servidor autoritativo; Resposta e cache.

Fato	Valor
Camada	Aplicação (camada 7)
Porta	53, UDP (TCP para respostas grandes)
Função	Nomes em endereços IP
Hierarquia	Raiz → TLD → autoritativo
Cache	Pelo TTL de cada registro
Definido em	RFC 1034, RFC 1035

DNS over HTTPS · DNS sobre HTTPS (DoH)

Segurança, camada 7 (Aplicação), 6 passos e 8 pacotes. Passos: O problema; Conexão TCP; Handshake TLS 1.3; Consulta cifrada; Resposta cifrada; O que fica visível.

Fato	Valor
Camada	Aplicação (camada 7)
Porta	443, a mesma do HTTPS
Transporte	TCP + TLS (ou QUIC no HTTP/3)
Formato	application/dns-message
Protege	Sigilo e integridade das consultas
Definido em	RFC 8484

DNSSEC · Extensões de Segurança do DNS

Segurança, camada 7 (Aplicação), 6 passos e 10 pacotes. Passos: Pede as assinaturas; Resposta assinada; Chaves da zona; DS na zona pai; Até a âncora; Resposta validada.

Fato	Valor
Camada	Aplicação (camada 7)
Porta	53, UDP com EDNS0 (respostas maiores)
Novos registros	RRSIG, DNSKEY, DS, NSEC/NSEC3
Garante	Origem e integridade, não sigilo

Copyright (©) 2026 Erik Pimenta

Fato	Valor
Confiança	Cadeia até a KSK da raiz
Definido em	RFC 4033, 4034, 4035

DHCP · Protocolo de Configuração Dinâmica de Host

Núcleo, camada 7 (Aplicação), 5 passos e 6 pacotes. Passos: Discover; Offer; Request; ACK; Renovação.

Fato	Valor
Camada	Aplicação (camada 7)
Portas	UDP 67 (servidor), 68 (cliente)
Troca	Discover, Offer, Request, ACK
Entrega	IP, máscara, gateway, DNS, prazo
Base	BOOTP (1985)
Definido em	RFC 2131, RFC 2132

ICMP · Protocolo de Mensagens de Controle da Internet

Núcleo, camada 3 (Rede), 5 passos e 8 pacotes. Passos: Ping; Resposta do eco; Traceroute: TTL 1; Traceroute: TTL 2; Destino inalcançável.

Fato	Valor
Camada	Rede (camada 3), dentro do IP
Protocolo IP	1 (ICMPv6 é 58)
Portas	Nenhuma: usa tipo e código
Ferramentas	ping, traceroute, Path MTU
Tipos comuns	0, 3, 8, 11
Definido em	RFC 792

NTP · Protocolo de Tempo de Rede

Aplicação, camada 7 (Aplicação), 5 passos e 6 pacotes. Passos: Sincronia acima; Cliente pergunta; Servidor responde; Calcula o ajuste; Próximas consultas.

Fato	Valor
Camada	Aplicação (camada 7)

Copyright (©) 2026 Erik Pimenta

Fato	Valor
Porta	UDP 123
Mensagem	48 bytes, 4 marcas de 64 bits
Hierarquia	Estratos 1 a 15 (16 = sem sincronia)
Precisão	~1 ms em rede local, ~10 ms na internet
Definido em	RFC 5905

Wi-Fi · Rede sem fio IEEE 802.11

Aplicação, camada 2 (Enlace), 6 passos e 13 pacotes. Passos: Beacon; Probe; Autenticação aberta; Associação; Aperto de mão de 4 vias; Endereço IP.

Fato	Valor
Camada	Enlace (camada 2)
Padrão	IEEE 802.11 (Wi-Fi 4 a 7)
Frequências	2,4, 5 e 6 GHz
Acesso ao meio	CSMA/CA: escuta antes de falar
Segurança	WPA2-PSK: 4 vias + AES-CCMP
Definido em	IEEE 802.11-2020 (inclui o 802.11i)

WPA3 · Wi-Fi Protected Access 3

Segurança, camada 2 (Enlace), 6 passos e 11 pacotes. Passos: Descoberta; SAE Commit; SAE Confirm; Associação; Aperto de mão de 4 vias; Por que supera o WPA2.

Fato	Valor
Camada	Enlace (camada 2)
Autenticação	SAE (Dragonfly, RFC 7664)
Cifra	AES-CCMP 128 (GCMP-256 no Enterprise 192)
Proteção de gerência	PMF obrigatório (802.11w)
Sigilo futuro	Sim: chave nova por conexão
Definido em	IEEE 802.11-2020, Wi-Fi Alliance (2018)

Copyright (©) 2026 Erik Pimenta

RADIUS · Remote Authentication Dial-In User Service

Segurança, camada 7 (Aplicação), 5 passos e 16 pacotes. Passos: Identidade; Pergunta ao RADIUS; Túnel PEAP; Aceite e chaves; Aperto de mão de 4 vias.

Fato	Valor
Camada	Aplicação (camada 7)
Portas	UDP 1812 (autenticação), 1813 (contabilidade)
Transporte	UDP; EAPOL no ar (802.1X)
Métodos EAP	PEAP-MSCHAPv2, EAP-TLS, EAP-TTLS
Confiança	Segredo compartilhado AP ↔ servidor
Definido em	RFC 2865, RFC 3579, IEEE 802.1X

Captive Portal · Portal cativo

Aplicação, camada 7 (Aplicação), 5 passos e 17 pacotes. Passos: Wi-Fi aberto e DHCP; Teste de conectividade; Página de login; Aceite dos termos; Internet liberada.

Fato	Valor
Camada	Aplicação (camada 7)
Descoberta	Opção DHCP 114, teste HTTP 204
Portas	80 (interceptada), 443 (portal)
Transporte	TCP; DHCP sobre UDP
Liberação	Regra no firewall por MAC/IP
Definido em	RFC 8908, RFC 8910, RFC 8952

SMTP · Protocolo Simples de Transferência de Correio

Aplicação, camada 7 (Aplicação), 5 passos e 28 pacotes. Passos: Conexão e saudação; STARTTLS; Autenticação e envelope; A mensagem; Entrega ao destino.

Fato	Valor
Camada	Aplicação (camada 7)
Portas	587 (envio), 25 (entre servidores), 465 (TLS direto)
Transporte	TCP
Formato	Comandos em texto + códigos de 3 dígitos

Copyright (©) 2026 Erik Pimenta

Fato	Valor
Segurança	STARTTLS, AUTH, SPF/DKIM/DMARC
Definido em	RFC 5321, RFC 6409, RFC 3207

POP3 · Post Office Protocol, versão 3

Aplicação, camada 7 (Aplicação), 5 passos e 20 pacotes. Passos: Conexão; Login; O que há na caixa; Baixar; Apagar e sair.

Fato	Valor
Camada	Aplicação (camada 7)
Portas	110 (texto), 995 (TLS)
Transporte	TCP
Respostas	+OK ou -ERR
Estados	AUTHORIZATION → TRANSACTION → UPDATE
Definido em	RFC 1939, RFC 2595

IMAP · Internet Message Access Protocol

Aplicação, camada 7 (Aplicação), 6 passos e 19 pacotes. Passos: Conexão; Login; Abrir a caixa; Ler e marcar; IDLE; Sair.

Fato	Valor
Camada	Aplicação (camada 7)
Portas	143 (texto/STARTTLS), 993 (TLS)
Transporte	TCP
Modelo	Mensagens ficam no servidor
Tempo real	IDLE (o servidor avisa)
Definido em	RFC 9051 (IMAP4rev2), RFC 2177

FTP · Protocolo de Transferência de Arquivos

Aplicação, camada 7 (Aplicação), 6 passos e 23 pacotes. Passos: Canal de controle; Login; Modo passivo; Canal de dados; Transferência; Encerrar.

Fato	Valor
Camada	Aplicação (camada 7)

Copyright (©) 2026 Erik Pimenta

Fato	Valor
Portas	21 (controle) + porta de dados
Transporte	TCP, duas conexões
Modos	Ativo (PORT) e passivo (PASV/EPSV)
Segurança	Nenhuma: tudo em texto aberto
Definido em	RFC 959, RFC 2428

SMB · Server Message Block

Aplicação, camada 7 (Aplicação), 6 passos e 17 pacotes. Passos: Conexão; NEGOTIATE; SESSION_SETUP; TREE_CONNECT; CREATE e READ; CLOSE.

Fato	Valor
Camada	Aplicação (camada 7)
Porta	TCP 445
Transporte	TCP (ou QUIC no SMB 3.1.1)
Formato	Binário, cabeçalho de 64 bytes
Segurança	Assinatura AES-CMAC, cifra AES-GCM
Definido em	MS-SMB2 (Microsoft)

SSH · Shell seguro

Segurança, camada 7 (Aplicação), 6 passos e 20 pacotes. Passos: Conexão TCP; Troca de versões; Negociação (KEXINIT); Troca de chaves ECDH; Autenticação por chave; Sessão e comandos.

Fato	Valor
Camada	Aplicação (camada 7)
Porta padrão	22
Transporte	TCP
Troca de chaves	curve25519-sha256 (ECDH)
Autenticação	Chave pública, senha ou certificado
Definido em	RFC 4251–4254, RFC 8731

Copyright (©) 2026 Erik Pimenta

TLS/SSL · Segurança da Camada de Transporte

Segurança, camada 6 (Apresentação), 6 passos e 14 pacotes. Passos: Conexão TCP; ClientHello; ServerHello e certificado; Finished do cliente; Dados da aplicação; Encerramento.

Fato	Valor
Camada	Apresentação (camada 6), entre o TCP e a aplicação
Portas	443 (HTTPS), 853 (DoT), 993 (IMAPS)
Transporte	TCP (DTLS e QUIC sobre UDP)
Handshake	1-RTT no TLS 1.3, 2-RTT no TLS 1.2
Obsoletos	SSL 2.0/3.0, TLS 1.0/1.1
Definido em	RFC 8446 (TLS 1.3)

HTTP Proxy · Proxy HTTP de saída

Aplicação, camada 7 (Aplicação), 5 passos e 18 pacotes. Passos: Conexão com o proxy; Autenticação (407); Pedido repassado; CONNECT para HTTPS; Túnel TLS.

Fato	Valor
Camada	Aplicação (camada 7)
Porta	3128 ou 8080 (configurável)
Transporte	TCP
Métodos	GET com URL absoluta, CONNECT
Autenticação	407 + Proxy-Authorization
Definido em	RFC 9110, RFC 9112

SOCKS5 · Proxy Socket Secure, versão 5

Aplicação, camada 5 (Sessão), 5 passos e 16 pacotes. Passos: Conexão com o proxy; Saudação; Usuário e senha; Pedido CONNECT; Dados repassados.

Fato	Valor
Camada	Sessão (camada 5)
Porta	1080
Transporte	TCP (UDP via UDP ASSOCIATE)
Autenticação	Nenhuma, usuário/senha, GSS-API

Copyright (©) 2026 Erik Pimenta

Fato	Valor
Destinos	IPv4, IPv6 ou nome de domínio
Definido em	RFC 1928, RFC 1929

Reverse Proxy · Proxy reverso

Aplicação, camada 7 (Aplicação), 5 passos e 20 pacotes. Passos: Verificação de saúde; TLS termina no proxy; Pedido repassado; Balanceamento; Resposta do cache.

Fato	Valor
Camada	Aplicação (camada 7)
Portas	443 público → 8080 interno
Funções	TLS, balanceamento, cache, WAF
Cabeçalhos	X-Forwarded-For, X-Forwarded-Proto, Forwarded
Exemplos	Nginx, HAProxy, Envoy, Traefik
Definido em	RFC 9110 (intermediários), RFC 7239

WireGuard · WireGuard

Segurança, camada 3 (Rede), 5 passos e 10 pacotes. Passos: Iniciação do handshake; Resposta do handshake; Dados pelo túnel; Keepalive; Rotação de chaves.

Fato	Valor
Camada	Rede (camada 3): túnel de pacotes IP
Porta	UDP 51820 (configurável)
Criptografia	Noise IK: Curve25519, ChaCha20-Poly1305, BLAKE2s
Handshake	1 ida e volta: 148 + 92 bytes
Chaves	Renovadas a cada 2 minutos
Definido em	Whitepaper WireGuard (NDSS 2017); Linux 5.6+

IPsec · Segurança do Protocolo de Internet

Segurança, camada 3 (Rede), 5 passos e 10 pacotes. Passos: Tráfego interessante; IKE_SA_INIT; IKE_AUTH; ESP pelo túnel; A resposta volta.

Fato	Valor
Camada	Rede (camada 3)

Copyright (©) 2026 Erik Pimenta

Fato	Valor
Portas	UDP 500 e 4500 (IKE), protocolo IP 50 (ESP)
Modos	Túnel (entre redes) e transporte (entre hosts)
Chaves	IKEv2 com Diffie-Hellman
Criptografia	AES-GCM, ChaCha20-Poly1305
Definido em	RFC 4301, RFC 4303, RFC 7296

Modbus · Modbus/TCP

Aplicação, camada 7 (Aplicação), 5 passos e 14 pacotes. Passos: Conexão TCP; Leitura de registradores; Escrita de um registrador; Exceção; Sem autenticação.

Fato	Valor
Camada	Aplicação (camada 7)
Porta	TCP 502 (802 com TLS)
Transporte	TCP, ou serial RTU/ASCII
Modelo	Cliente/servidor, registradores de 16 bits
Segurança	Nenhuma autenticação nem criptografia
Definido em	Modbus Application Protocol v1.1b3

Copyright (©) 2026 Erik Pimenta

07 Qualidade e segurança

7.1 Testes

Comando	O que confere
npm run typecheck	TypeScript em modo estrito, incluindo testes e configurações
npm test	107 testes: catálogo (33, com 8, 16 e 9 por categoria), validação de cada protocolo, montagem de todos os pacotes, checksums IPv4, TCP e UDP, continuidade das camadas, determinismo, rotas e linha do tempo
npm run test:e2e	Interface com Playwright: abas e contagens, reprodução, inspetor de 75 bytes, busca, idioma e tema salvos e política de segurança

7.2 Integração contínua

O workflow **CI** roda em cada push e pull request: auditoria de dependências (`npm audit`), TypeScript, testes unitários, build e Playwright. O workflow **GitHub Pages** repete os testes antes de publicar. O Dependabot propõe atualizações de npm toda semana e de GitHub Actions todo mês.

7.3 Segurança

- **Política de segurança de conteúdo** no build: `default-src 'self'` , sem scripts ou estilos externos, `object-src 'none'` , `base-uri 'self'` e `form-action 'none'` .
- **Sem HTML a partir de texto:** nenhum uso de `innerHTML` , `eval` ou `new Function` ; estilos dinâmicos usam o CSSOM, que a política permite.
- **Preferências validadas:** o que vem do `localStorage` é conferido campo a campo e qualquer erro de leitura é ignorado.
- **Sem dados pessoais:** não há servidor, cookies, formulários ou rastreadores.
- **Links externos** abrem com `rel="noopener noreferrer"` .
- **Dependências mínimas** e versões exatas no `package.json` , com lockfile.

7.4 Acessibilidade

- Todos os controles são botões de verdade, com rótulo e estado (`aria-pressed` , `aria-expanded` , `aria-selected`).
- Navegação completa pelo teclado, com foco visível e atalhos que não interferem em campos de texto.
- As vistas em SVG têm rótulo descritivo.
- Respeito a `prefers-reduced-motion` .

Copyright (©) 2026 Erik Pimenta

- As cores da paleta foram escolhidas com contraste para os dois temas.

Copyright (©) 2026 Erik Pimenta

08 Personalização

8.1 Cores e temas

As cores da paleta do Bento ficam em tokens CSS no começo de `src/css/estilo.css`, um bloco para o tema claro e outro para o escuro (`data-tema` no elemento `html`).

Token	Uso
--mesa, --caixa, --caixa-2, --caixa-3	Fundo da página e dos painéis
--parede, --parede-forte	Bordas finas e bordas de controles
--texto, --texto-2, --texto-3, --texto-4	Hierarquia do texto
--tampa, --tampa-texto	Botão principal e itens ativos
--c-anil, --c-oliva, --c-ouro, --c-brasa, --c-neutra	Cores dos tipos de pacote e das camadas

Para ajustar uma cor, troque o valor do token nos dois blocos; toda a interface acompanha.

8.2 Tipografia

Títulos em Bricolage Grotesque condensada, texto em Instrument Sans e dados técnicos (IPs, bytes, tabela) em IBM Plex Mono. As fontes são servidas pelo próprio site via Fontsource e as variáveis `--fonte-titulo`, `--fonte-texto` e `--fonte-mono` concentram a escolha.

8.3 Marca

O nome, a versão, o endereço do repositório e a grade de pixels da logo ficam em `src/marca.ts`. O ícone do site é `public/logo.svg` e o letreiro em pixels é gerado em `src/ui/icones.ts`.

Copyright (©) 2026 Erik Pimenta

09 Solução de problemas

9.1 Problemas comuns

Sintoma	Causa provável e solução
Página em branco depois de publicar	O GitHub Pages ainda não está em GitHub Actions. Em Settings, Pages, escolha GitHub Actions e rode o workflow de novo.
O workflow de Pages falha na etapa de deploy	O ambiente github-pages precisa existir e permitir a branch main. Ele é criado ao escolher GitHub Actions como fonte.
npm ci falha	Versão do Node.js abaixo de 22. Use nvm use ou fnm use na pasta do projeto.
As animações não aparecem	O sistema está com redução de movimento ativa. O Ocelo respeita essa escolha e mostra os pacotes já no destino.
A preferência de tema ou idioma não fica salva	O navegador bloqueia o localStorage (modo anônimo restrito). A aplicação continua funcionando, só sem lembrar a escolha.
O Playwright não encontra o navegador	Rode npx playwright install chromium. Para usar um Chrome já instalado, defina OCELO_CHROME com o caminho do executável.
O teste de validação falha depois de editar um protocolo	A mensagem mostra o protocolo e o item: texto vazio, nó inexistente, faixa ausente, tipo sem legenda ou quadro maior que 1600 bytes.

Copyright (©) 2026 Erik Pimenta